

Building Behavioral Tasks with



Why computer tasks?

- Precision
 - presentation of stimuli
 - record of responses
- Standardization
- Replicability
- Flexibility

Phonautograph (sound-self-graph)

Noematachograph (thought-speed-graph)

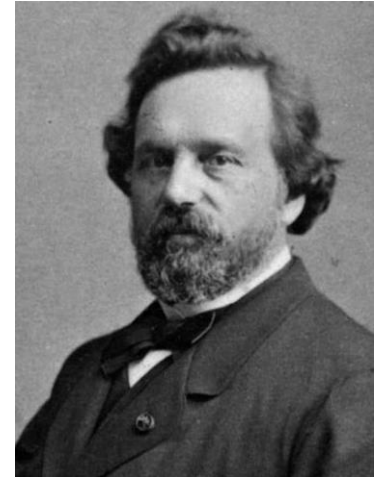
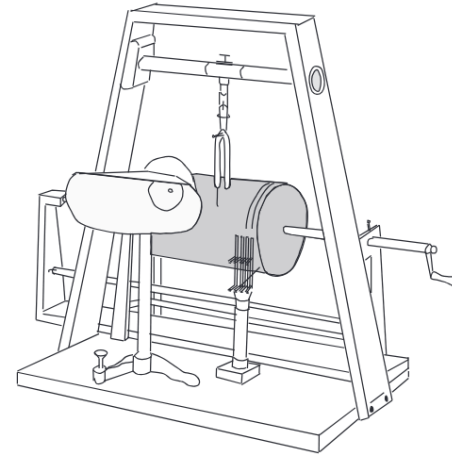
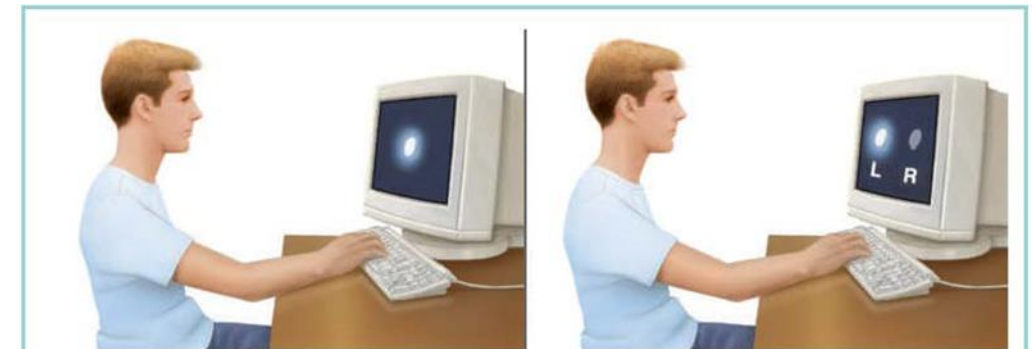


Fig. 1. Stylized versions of the phonautograph and noematachograph, and the noematachograph from Donders (1868) showing the speech waves of two persons (P), the sound wave of a tuning fork (T), and vertical marks indicating the speech onsets (A and B) of the persons.



(a) Press J when light goes on.

(b) Press J for left light, K for right.

Figure 1.2 A modern version of Donders's (1868) reaction time experiment: (a) the simple reaction time task and (b) the choice reaction time task. In the simple reaction time task, the subject pushes the J key when the light goes on. In the choice reaction time task, the subject pushes the J key if the left light goes on and the K key if the right light goes on. The purpose of Donders's experiment was to determine how much time it took to decide which key to press in the choice reaction time task. © Cengage Learning

Why PsychoPy?

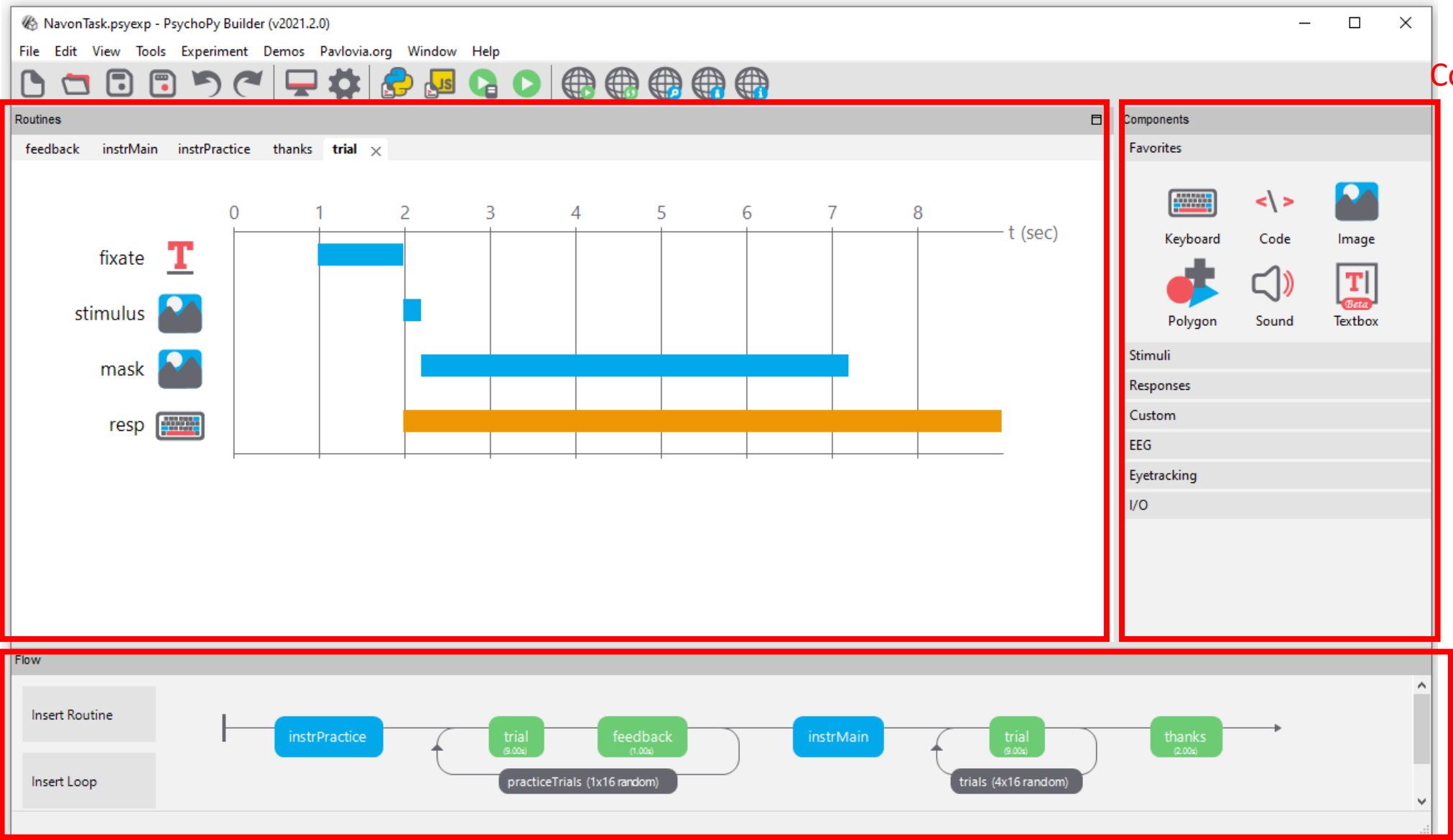
- Commonly used programs are limited
- Different computer programs are available
- But PsychoPy is:
 - Free
 - Open-sourced, open access
 - Graphical user interface (GUI)
 - Python-based



Builder

Routines

Components



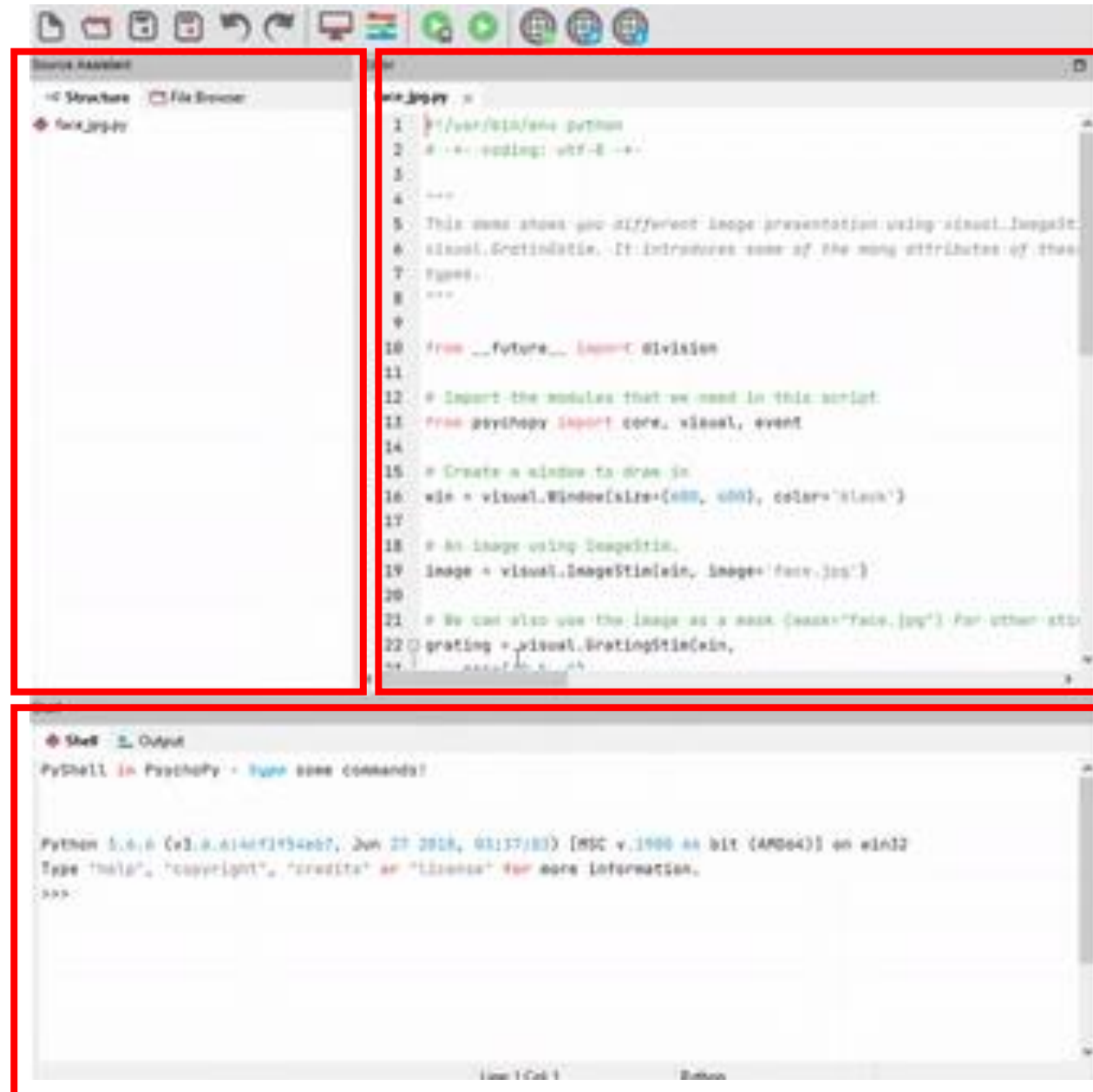
Flow

Coder

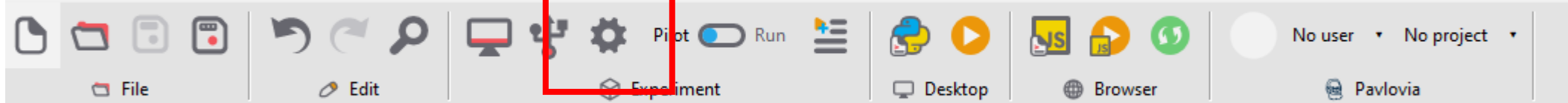
Source assistant (Directory)

Editor (Script)

Shelf (Console)



Let's build a task!



Routines

SRT Properties

Basic Screen Audio Online Eyetracking Input Data

Experiment name: SRT

Experiment version:

Run mode: Piloting

Use PsychoPy version: latest

Enable escape key: ☒

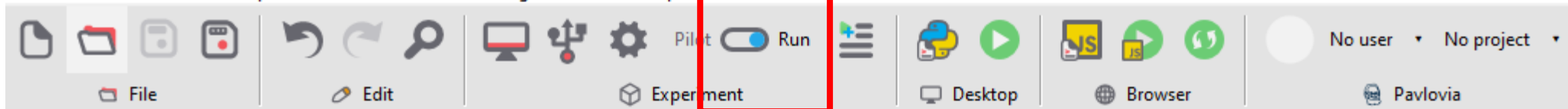
Enable 'rush' mode: ☐

Show info dialog: ☐

participant: \$ f"{randint(0, 999999):06.0f}"

Experiment info: session: 001

Add item

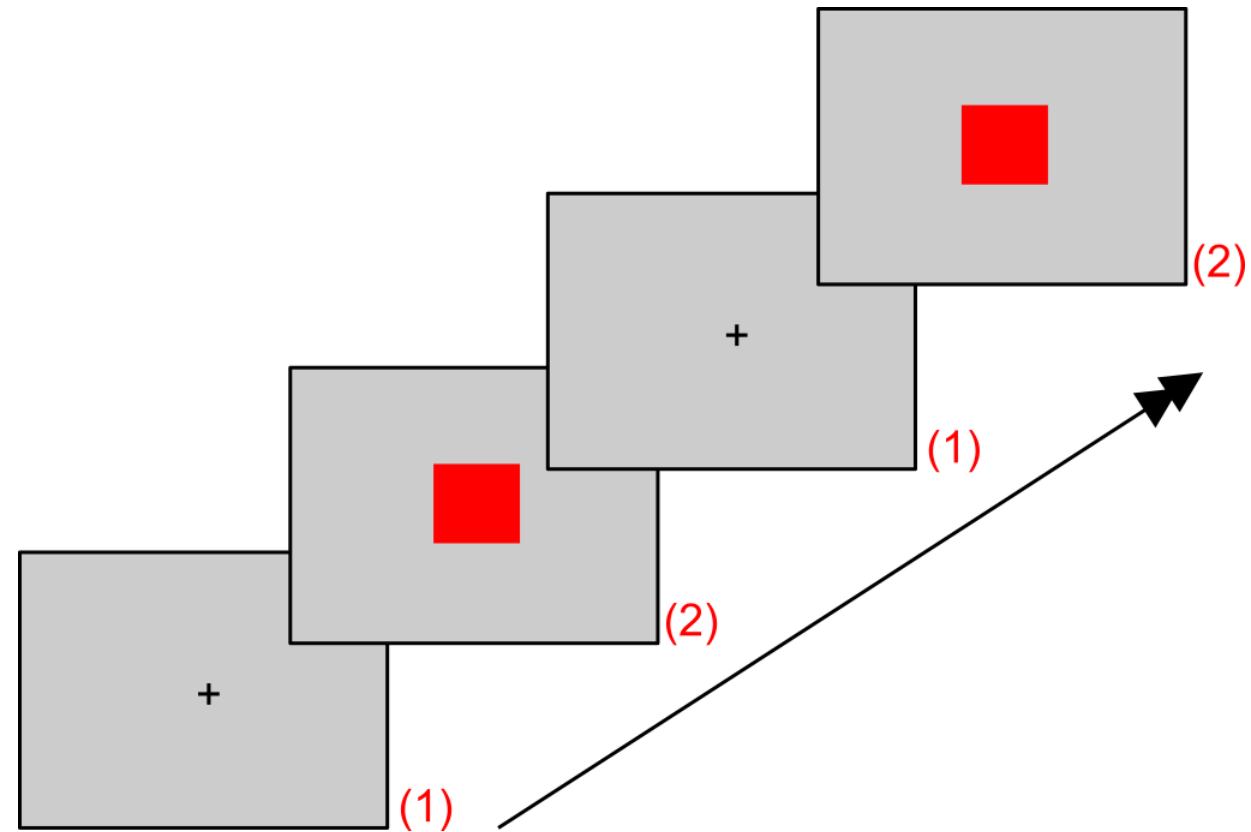


Routines

Enable full screen

SRT – part one

A dot and a keypress



target Properties

Basic Layout Appearance **Texture** Data Testing

Name **informative name**

Start 0.0
Expected start (s)

Stop
Expected duration (s) **= unlimited time**

Shape

Disable Component ☐

Help OK Cancel

Components

+ Get more...

Favorites

Image Keyboard Mouse

Slider Sound Text

Stimuli

Aperture **Polygon** Dots

Grating Image Movie

Beta Panorama *Beta* Progress Sound

Text Textbox

Responses

Custom

Eyetracking

EEG

target Properties

Basic Layout Appearance Texture Data Testing

Size [w,h] \$ (0.01, 0.01) = 0.01 of screen height constant

Position [x,y] \$ (0, 0) constant

Spatial units from exp settings

Anchor center

Orientation \$ 0 constant

Draggable? ☐

Help OK Cancel

Components

+ Get more...

Favorites

Image Keyboard Mouse

Slider Sound Text

Stimuli

Aperture Polygon Dots

Grating Image Movie

Panorama Progress Sound

Text Textbox

Responses

Custom

Eyetracking

EEG

trial_key_resp Properties

Basic Data

Name trial_key_resp Name that you can remember

Start \$ time (s) 0.0
Expected start (s)

Stop \$ duration (s) Empty = unlimited time
Expected duration (s)

Force end of Routine ☒

Register keypress on... press

Allowed keys \$ 'space' Empty = any key constant

Disable Component ☐

Help OK Cancel

Components

+ Get more...

Favorites

Image Keyboard Mouse

Slider Sound Text

Stimuli

Aperture Polygon Dots

Grating Image Movie

Panorama Progress Sound

Text Textbox

Responses

Custom

Eyetracking

EEG

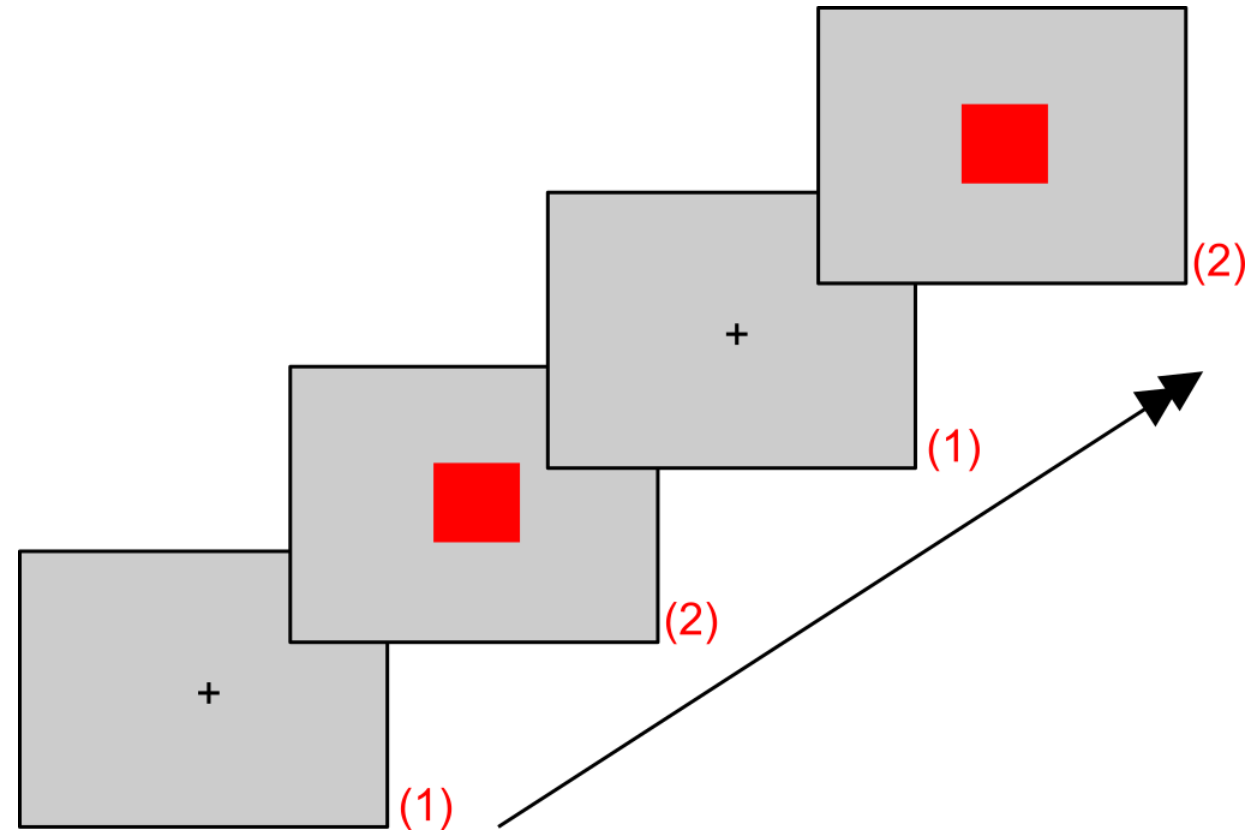
SRT.psyexp - PsychoPy Builder (v2026.1.3)

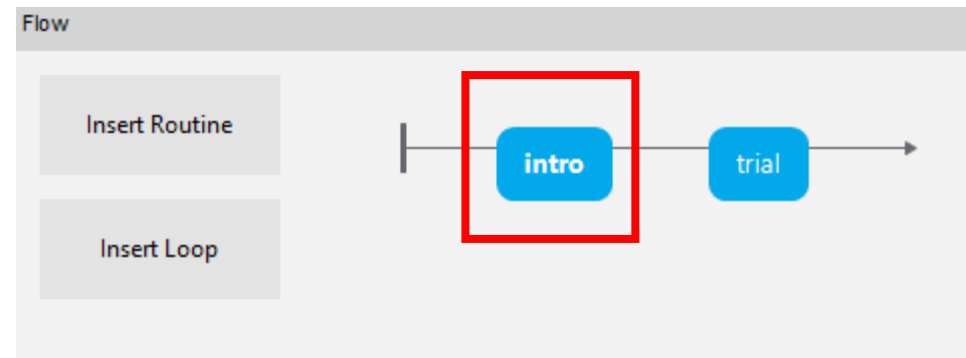
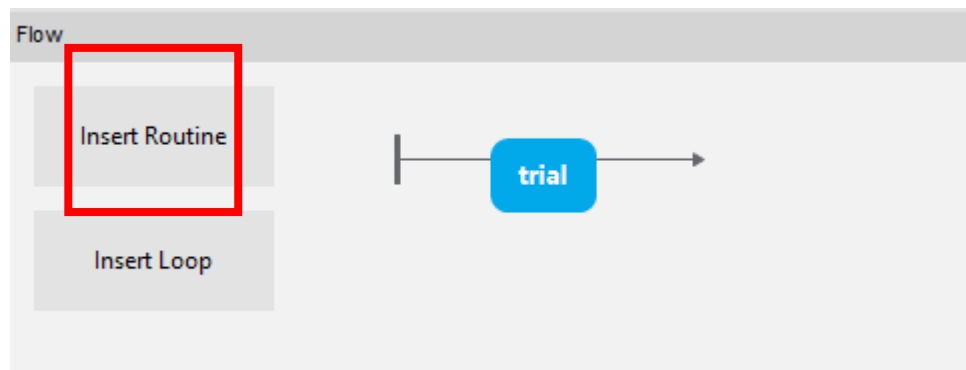
File Edit View Tools Experiment Demos Pavlovia.org Window Help



SRT – part two

Instructions, a dot and a keypress, end





text Properties

Basic Layout Appearance **Formatting** Data Testing

Name

Start $\$$ time (s) 0.0
Expected start (s)

Stop $\$$ duration (s)
Expected duration (s)

Disable Component ☐

Instructions
Your task is to press the space bar when you see a dot.

Text

Help OK Cancel

Components

+ Get more...

Favorites

- Image
- Keyboard
- Mouse
- Slider
- Sound
- Text**

Stimuli

- Aperture
- Polygon
- Dots
- Grating
- Image
- Movie
- Panorama**
- Progress**
- Sound
- Text**
- Textbox

Responses

- Custom
- Eyetracking
- EEG

key_resp Properties

Basic Data

Name intro_key_resp

Start \$ time (s) 0.0
Expected start (s)

Stop \$ duration (s)
Expected duration (s)

Force end of Routine ☒

Register keypress on... press

Allowed keys \$ 'space' constant

Disable Component ☐

Help OK Cancel

Components

+ Get more...

Favorites

Image Keyboard Mouse

Slider Sound Text

Stimuli

Aperture Polygon Dots

Grating Image Movie

Panorama Progress Sound

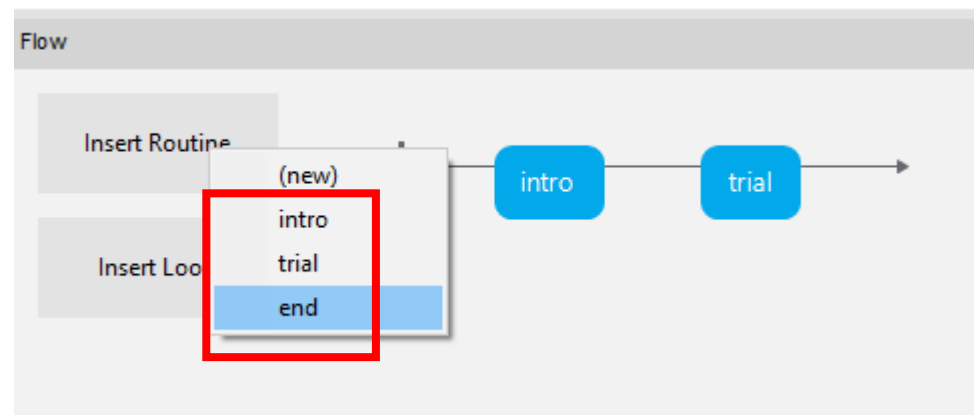
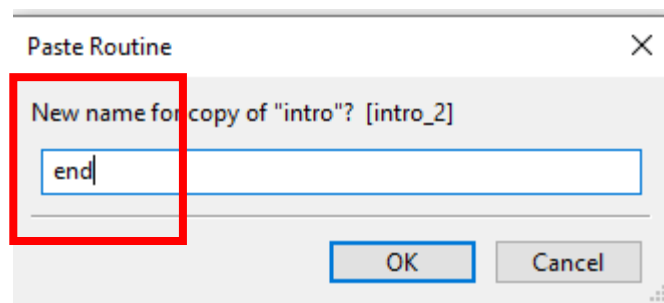
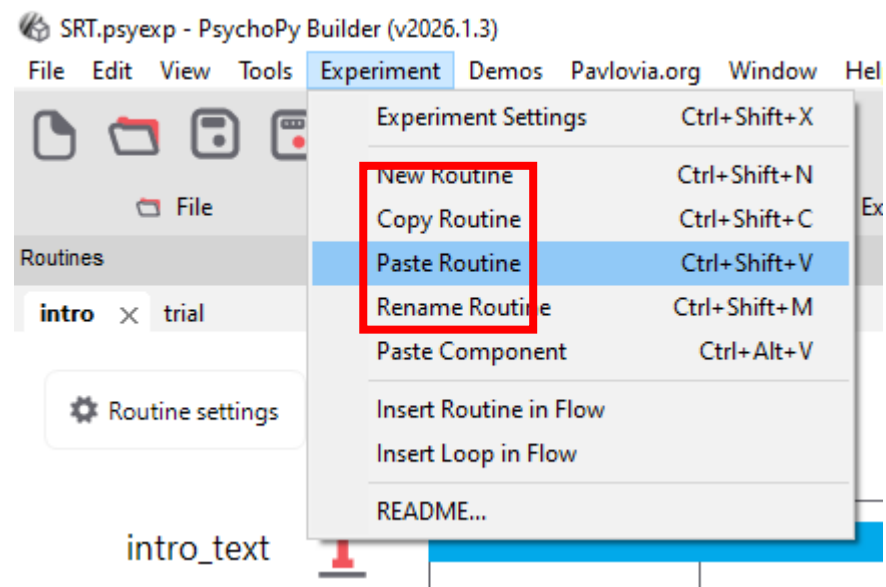
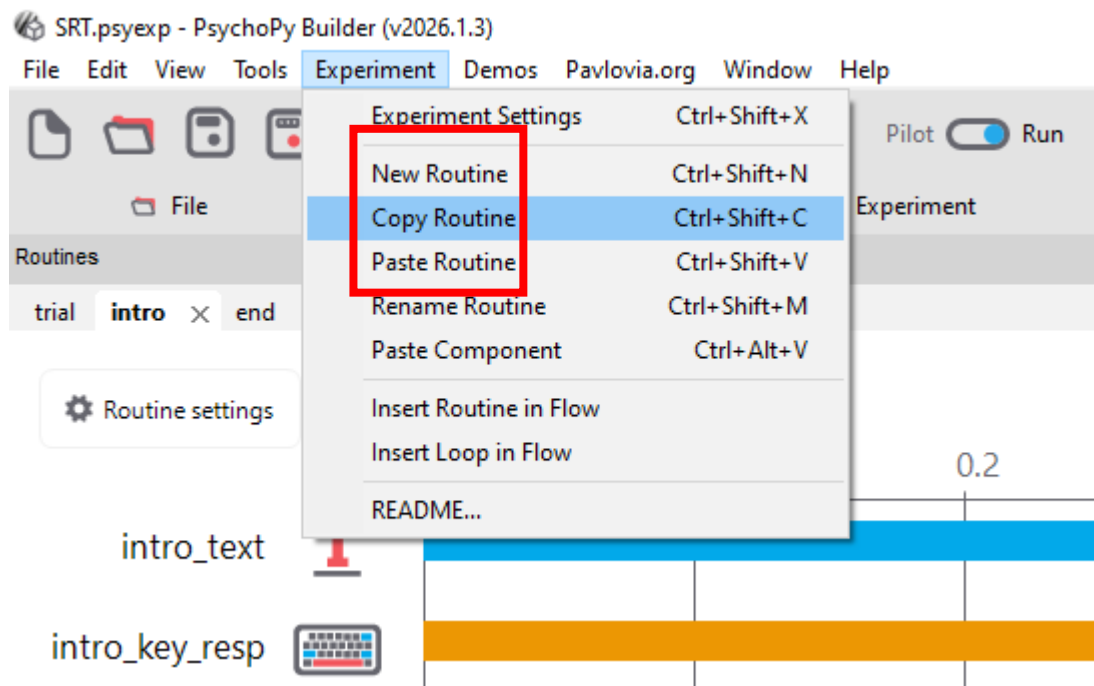
Text Textbox

Responses

Custom

Eyetracking

EEG



intro_text_2 Properties

Basic Layout Appearance Formatting Data Testing

Name

Start \$ 0.0
Expected start (s)

Stop \$
Expected duration (s)

Disable Component ☐

Text

Help OK Cancel

intro_key_resp_2 Properties

Basic Data

Name

Start \$ 0.0
Expected start (s)

Stop \$
Expected duration (s)

Force end of Routine ☒

Register keypress on...

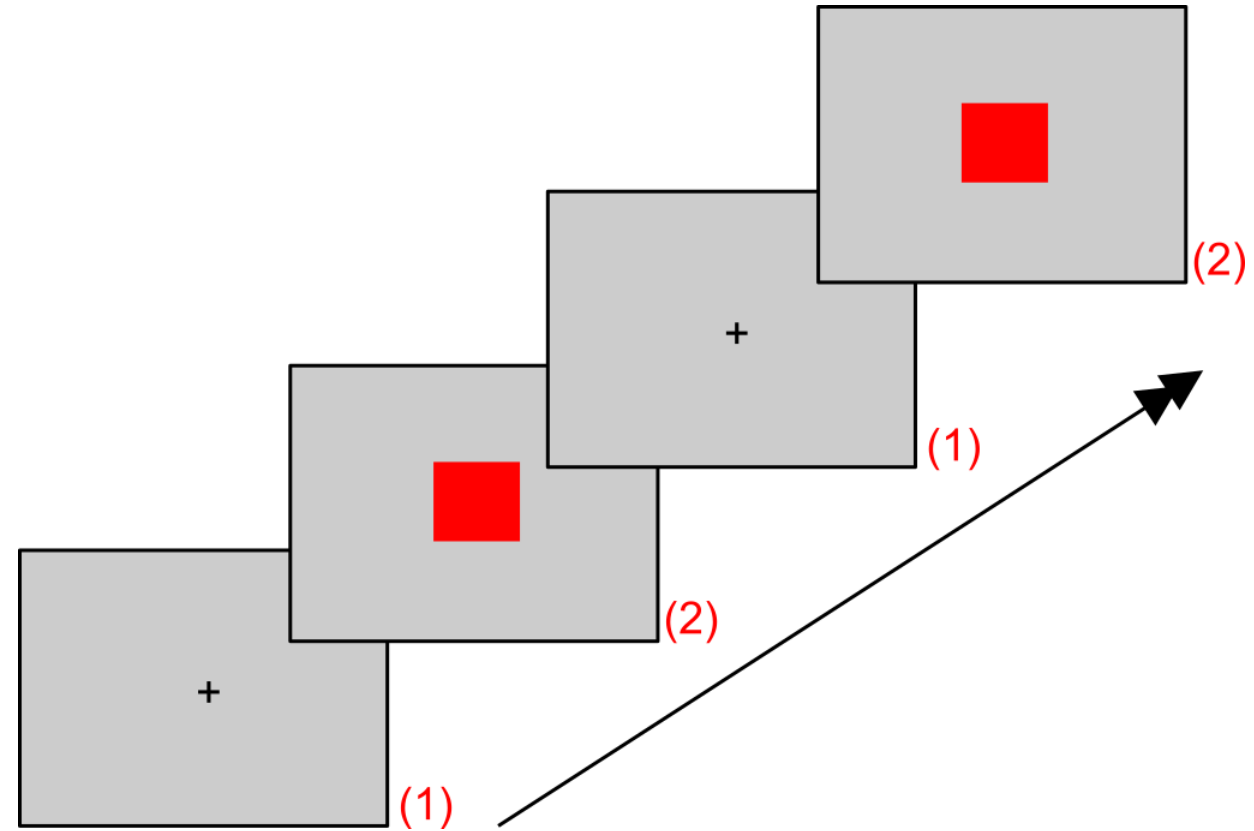
Allowed keys \$

Disable Component ☐

Help OK Cancel

SRT – part three

Instructions,
[random ISI, a dot and a keypress] <-repeat this,
end





New Routine

New Routine name:

Routine Template:

OK Cancel

ISI Properties

Basic Data Custom

Name ISI_random

Start \$ time (s) 0.0
Expected start (s)

Stop \$ duration (s) \$random()
Expected duration (s)

Disable Component ☐

\$: read as Python code

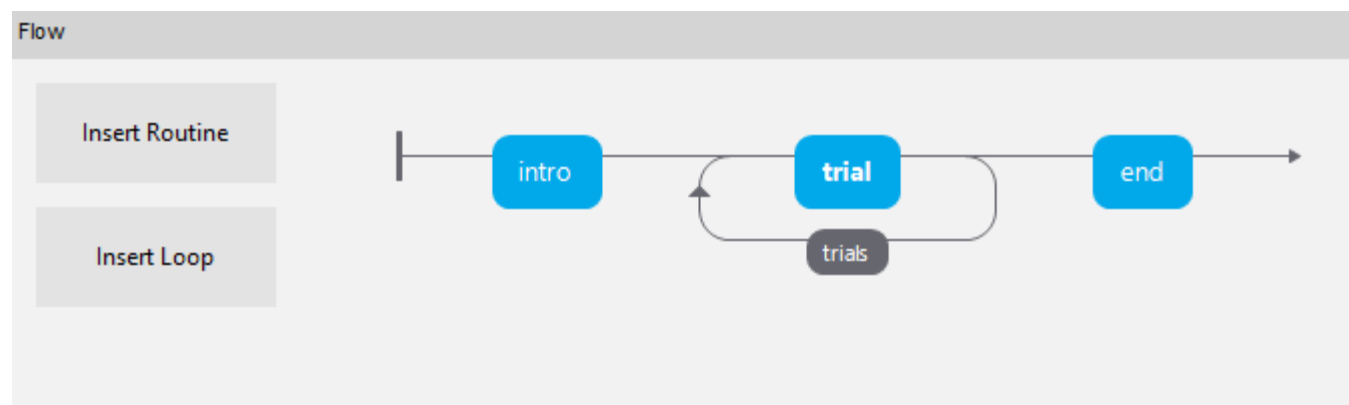
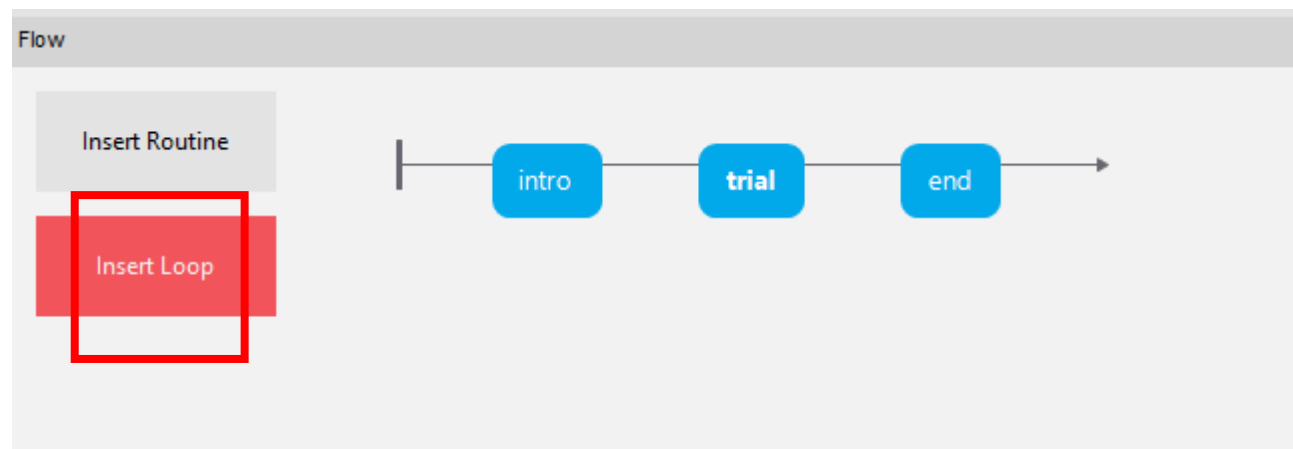
Help OK Cancel

Responses

Brush Button Button Box
Camera Form Joy Buttons
Joystick Keyboard Microphone
Mouse Slider Sound Sensor
Textbox Pavlovian Survey

Custom

Code Resource Manager Static
Variable Counterbalance



Loop Properties

Name: trials

Loop type: random

Is trials: ☒

Num. repeats: 5

Selected rows:

Random seed:

Conditions:

No parameters set

Help OK Cancel

ISI_random Properties

Basic Data Custom

Name ISI_random

Start \$ time (s) 0.0
Expected start (s)

Stop \$ duration (s) $\$random()+1$
Expected duration (s)

Disable Component ☐

Help OK Cancel

trial_target Properties



Basic

Layout

Appearance

Texture

Data

Testing

Name

trial_target

Start

\$ time (s)

0

Expected start (s)

Stop

\$ duration (s)

Expected duration (s)

Shape

Circle

Disable Component



Help

OK

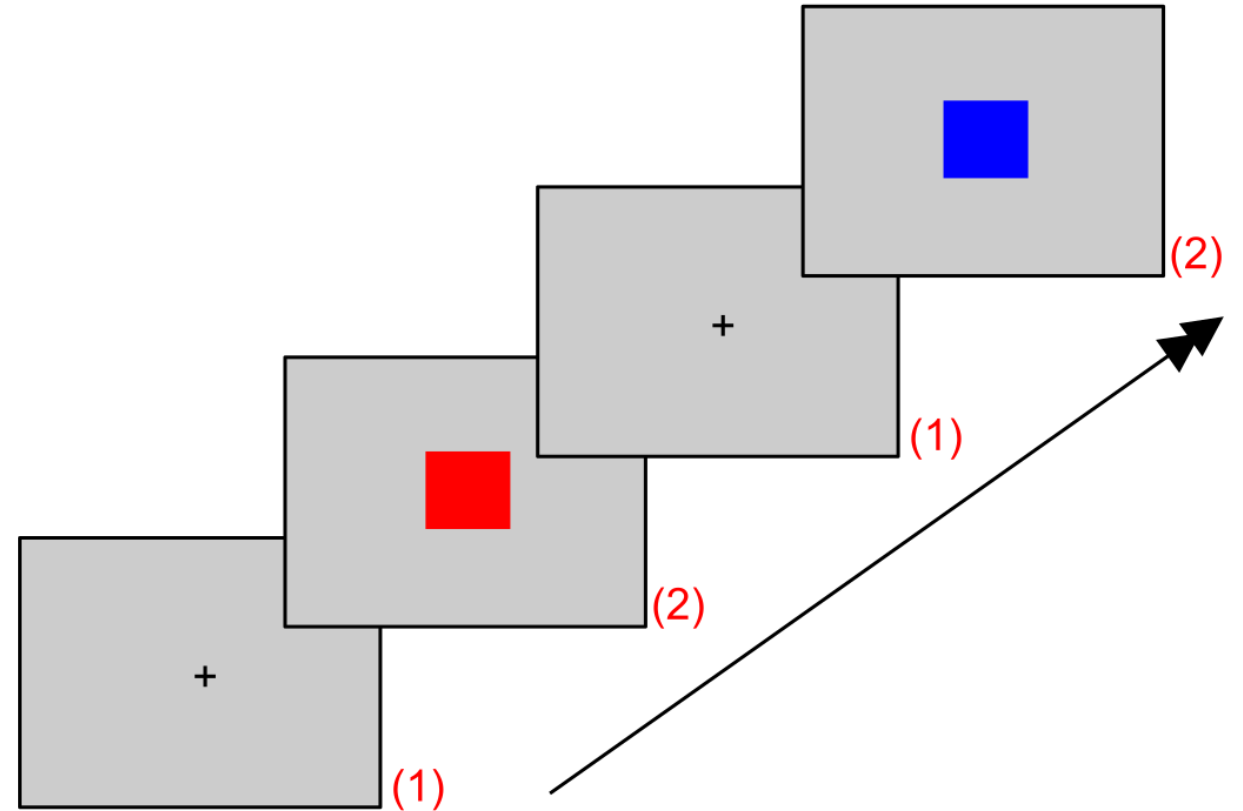
Cancel

Challenge #1

- Present something on screen somewhere, for some period of time, and require some responses
 - Stimulus:
 - Text
 - Image
 - Video
 - Response:
 - Keypress
 - Mouse click
 - Slider

CRT

Instructions,
[random ISI, either stimulus A or B, and a keypress] <-repeat this,
end



https://www.neurobs.com/manager/content/docs/psychlab101_experiments/Choice%20Reaction%20Time/description.html

trial_key_resp Properties



Basic

Data

Name

trial_key_resp

Start

\$ time (s) ▾

0

Expected start (s)

Stop

\$ duration (s) ▾

Expected duration (s)

Force end of Routine



Register keypress on...

press ▾

Allowed keys

'left', 'right'

constant ▾

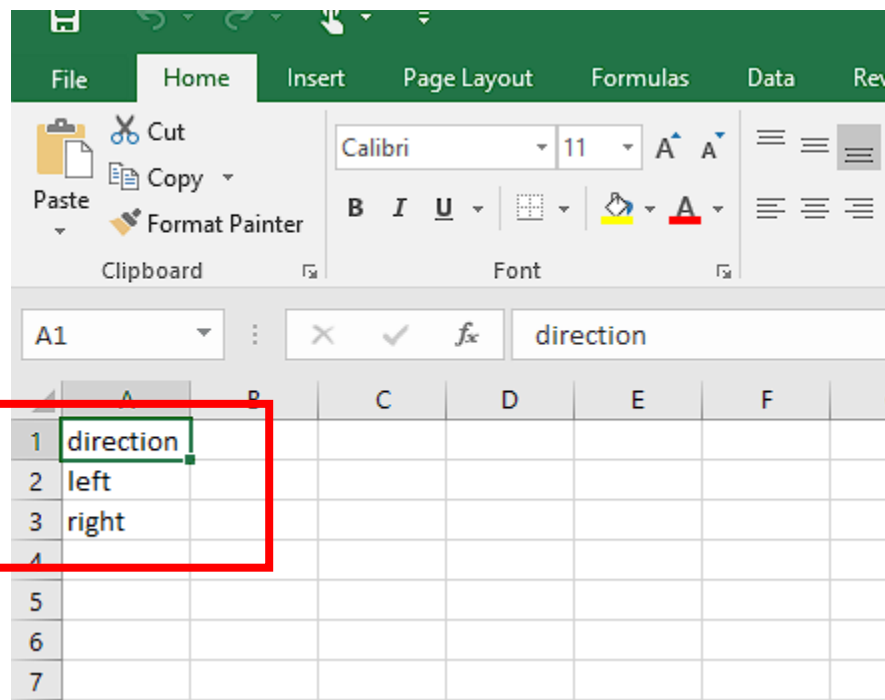
Disable Component



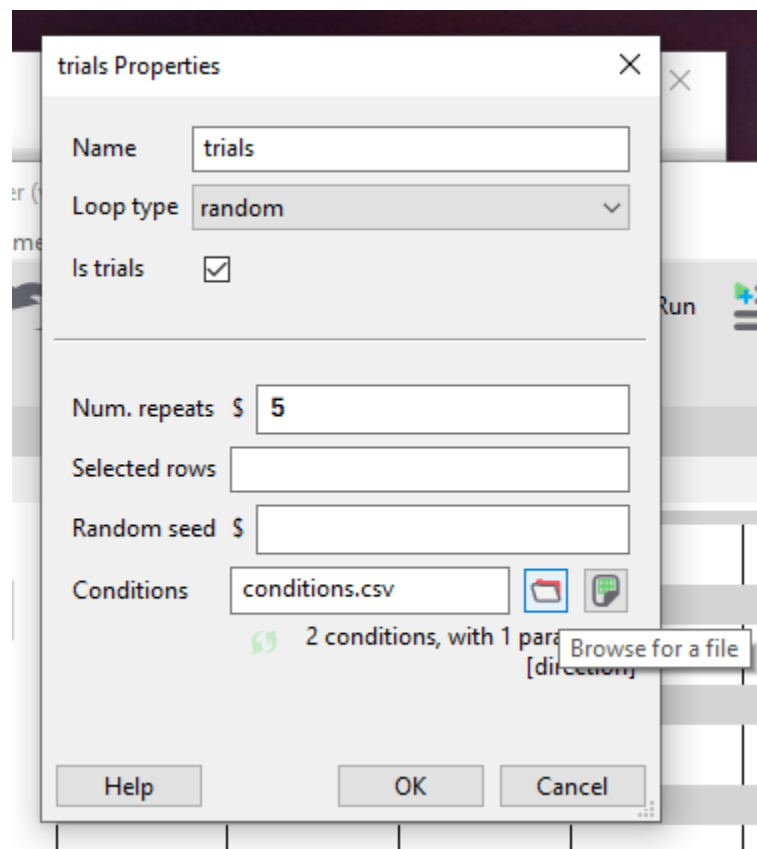
Help

OK

Cancel



conditions.csv



SRT building - PowerPoint

text Properties

Basic | Layout | Appearance | Formatting | Data | Testing

Name: text

Start: \$ time (s) 0.0
Expected start (s)

Stop: \$ duration (s)
Expected duration (s)

Disable Component ☐

\$direction

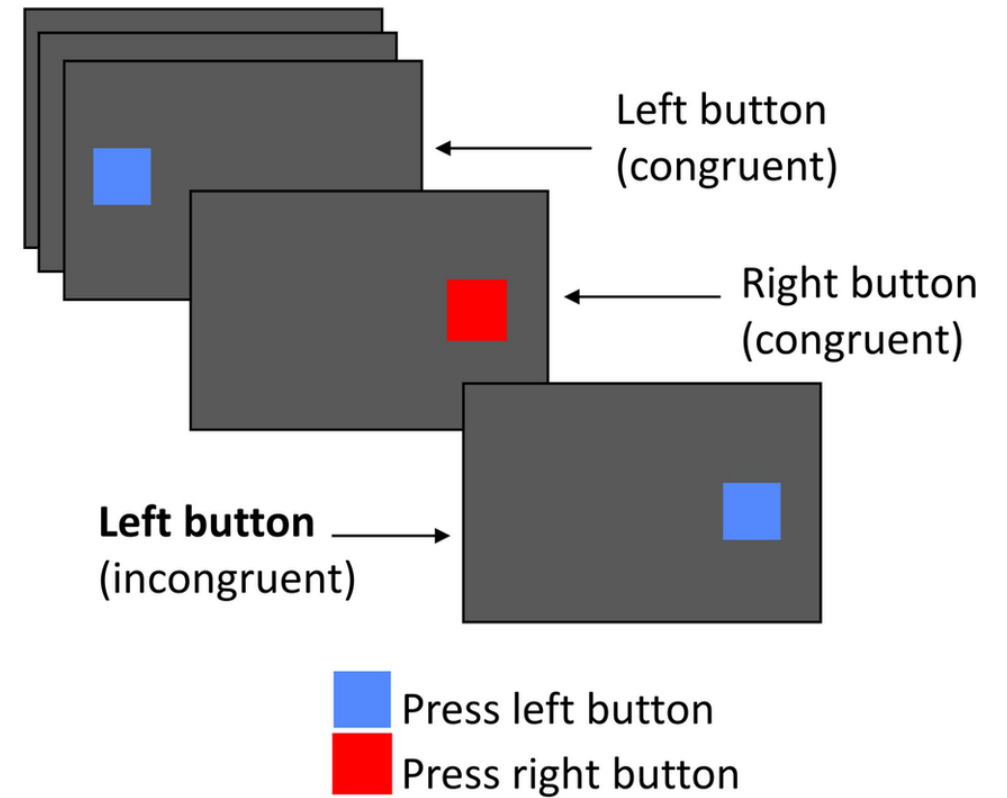
Text

set every repeat
constant
set every repeat
set every frame
set during: ISI.ISI_random

Help OK Cancel

Simon task

Instructions,
[random ISI, either stimulus “left” or “right” on either left or right, and a keypress]
<-repeat,
end



	A	B
1	direction	position
2	left	$[-0.25, 0]$
3	right	$[-0.25, 0]$
4	left	$[0.25, 0]$
5	right	$[0.25, 0]$

	Text	Position
Condition 1	Left	Left
Condition 2	Right	Left
Condition 3	Left	Right
Condition 4	Right	Right

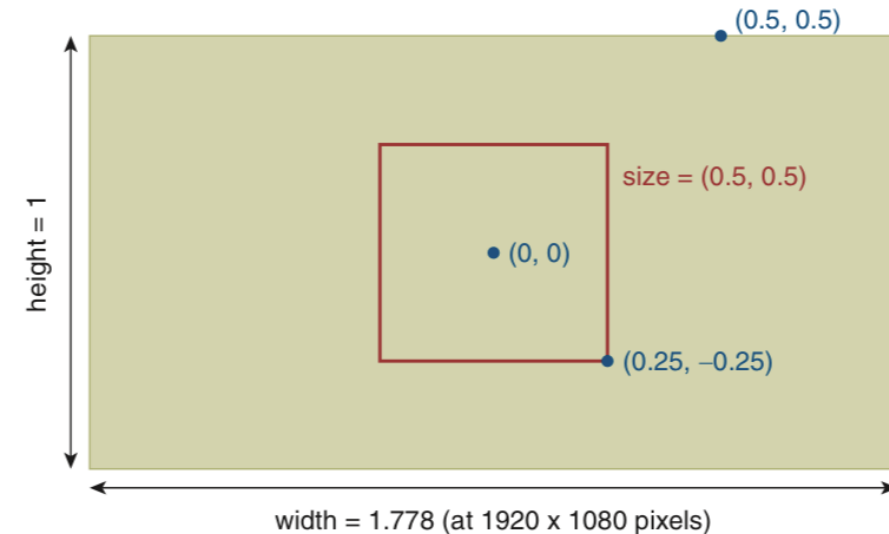
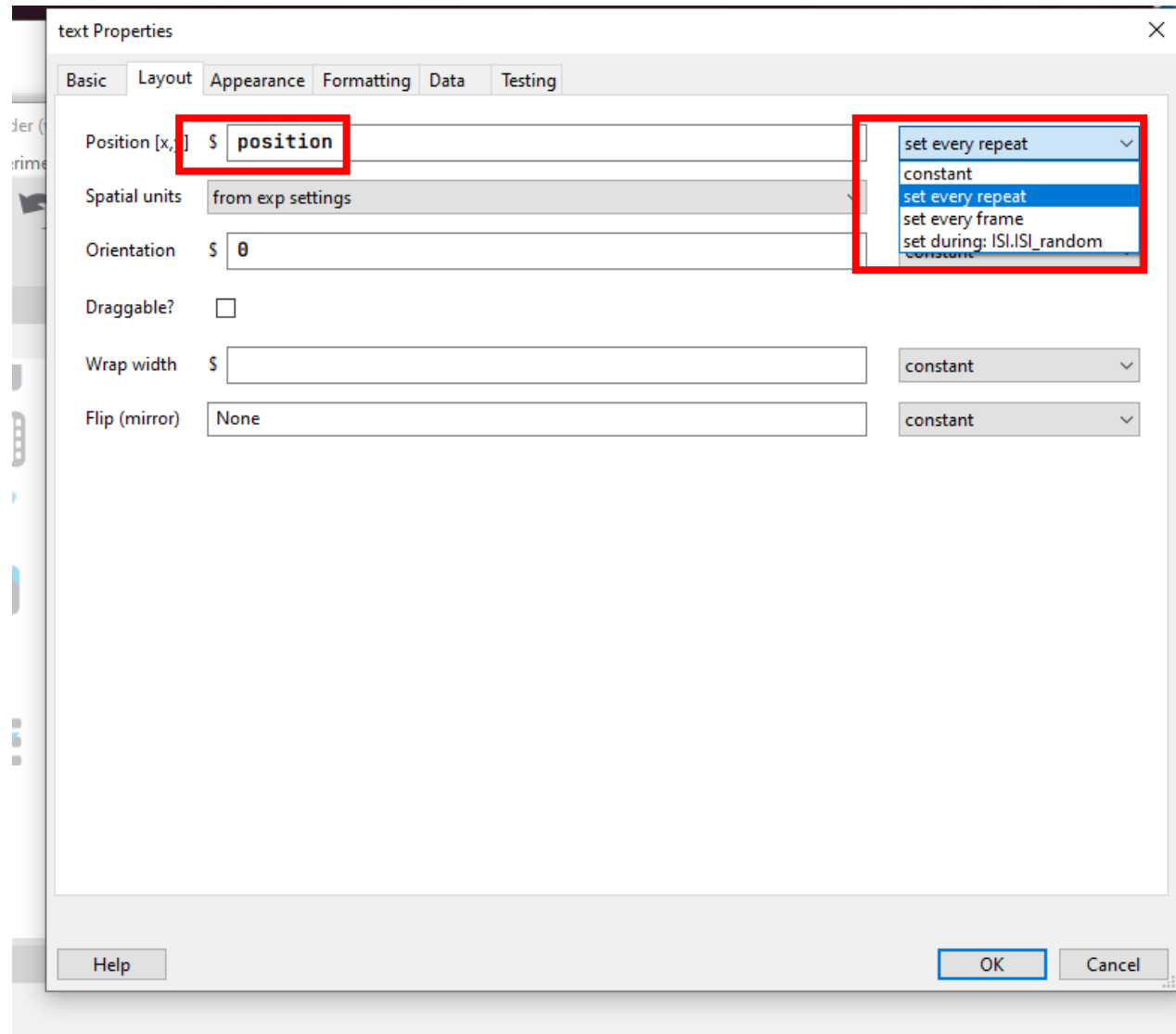


Figure 13.2 An illustration of height units assuming a wide-screen aspect ratio monitor (e.g. 1920×1080). The intuitive thing in this coordinate system is that the size of objects seems very sensible: when you ask for a size of $(0.5, 0.5)$ you get a square object whose width and height are half the height of the screen. What is less intuitive is that, to know how wide the screen is, you need to know its aspect ratio. It also doesn't seem quite as intuitive that placing an object on the top edge of the screen (for instance) is achieved by giving it a y position of 0.5 (because this is a half-screen distance up from the center of the screen).

Peirce, J., Hirst, R., & MacAskill, M. (2022). *Building experiments in PsychoPy* (2nd ed.). Sage Publications.



Routines

intro trial end ISI ×

⚙ Routine settings

ISI_dot

0 1 2

--	--	--

Challenge #2

- Present something on screen somewhere, for some period of time, and require some responses, with different conditions
 - Oddball task
 - Go/No-Go Task
 - Flanker task?
 - Stroop task?

Data

Data

Stimulus parameters

Response parameters

Routine parameters

[illegible]

Log

456306_CRT_2026-05-27_14h50.49.742 - Notepad

File Edit Format View Help

```
1.0344 INFO Loaded monitor calibration from ['2023_11_01 15:39']
1.1488 INFO Using renderer 'AMD Radeon RX 6500M' for graphics
1.1488 INFO OpenGL version supported by driver is 4..
1.1926 EXP Created window1 = Window(allowGUI=True, allowStencil=False, autoLog=True, backendConf=UNKNOWN, backgroundFit=<method-wrapper '__getattr__' of attributeSetter object at 0x0000023898F307C0>, backgroundImage=<method-wra
1.1927 EXP window1: backgroundImage = ''
1.1927 EXP window1: backgroundFit = 'none'
1.4203 EXP window1: Attempting to measure frame rate of screen (0) ...
1.4204 EXP window1: recordFrameIntervals = False
1.7960 EXP window1: recordFrameIntervals = True
1.8616 WARNING t of last frame was 33.22ms (=1/30)
1.8950 WARNING t of last frame was 33.36ms (=1/29)
1.9283 WARNING t of last frame was 33.32ms (=1/30)
1.9617 WARNING t of last frame was 33.40ms (=1/29)
1.9952 WARNING Multiple dropped frames have occurred - I'll stop bothering you about them!
5.1328 EXP window1: recordFrameIntervals = False
5.1329 EXP Screen (0) actual frame rate measured at 29.96Hz
5.2285 INFO keyboard.Keyboard is using ptb backend.
5.2285 EXP CRT: status = STARTED
5.2510 EXP Created intro_text = TextStim(__class__=<class 'psychopy.visual.text.TextStim'>, alignHoriz=method-wrapper(...), alignText='center', alignVert=method-wrapper(...), anchorHoriz='center', anchorVert='center', antialias=True
5.2537 EXP Created ISI_dot = ShapeStim(__class__=<class 'psychopy.visual.shape.ShapeStim'>, anchor=('center', 'center'), autoDraw=False, autoLog=True, closeShape=True, color=method-wrapper(...), colorSpace='rgb', contrast=None, dept
5.2552 EXP Created trial_target = ShapeStim(__class__=<class 'psychopy.visual.shape.ShapeStim'>, anchor=('center', 'center'), autoDraw=False, autoLog=True, closeShape=True, color=method-wrapper(...), colorSpace='rgb', contrast=None,
5.2556 EXP Created text = TextStim(__class__=<class 'psychopy.visual.text.TextStim'>, alignHoriz=method-wrapper(...), alignText='center', alignVert=method-wrapper(...), anchorHoriz='center', anchorVert='center', antialias=True, auto
5.2576 EXP Created end_text = TextStim(__class__=<class 'psychopy.visual.text.TextStim'>, alignHoriz=method-wrapper(...), alignText='center', alignVert=method-wrapper(...), anchorHoriz='center', anchorVert='center', antialias=True,
0.0061 INFO Keyboard events cleared
0.0061 EXP intro_text: autoDraw = True
0.8192 EXP Device response: <KeyPress from None: t=0.7905956000322476, value=space, duration=None>
0.8251 EXP Imported conditions.csv as conditions, 4 conditions, 2 params
0.8254 EXP New trial (rep=0, index=0): <Trial 0 (0 in rep 0) data={'direction': 'left', 'position': [-0.25, 0], 'thisN': 0, 'thisTrialN': 0, 'thisRepN': 0}>
0.8410 EXP intro_text: autoDraw = False
0.8410 EXP ISI_dot: autoDraw = True
0.8762 EXP Device response: <KeyPress from None: t=0.7905956000322476, value=space, duration=0.061850199941545725>
0.8777 DATA Keypress: space
2.1766 INFO Keyboard events cleared
2.1763 EXP ISI_dot: autoDraw = False
2.1763 EXP ISI_dot: autoDraw = False
2.1763 EXP text: height = np.float64(0.05)
2.1763 EXP text: height = np.float64(0.05)
2.1763 EXP text: height = np.float64(0.05)
2.1763 EXP text: text = 'left'
2.1763 EXP trial_target: autoDraw = True
2.1763 EXP text: autoDraw = True
2.7112 EXP Device response: <KeyPress from None: t=2.7009260000195354, value=left, duration=None>
2.7118 EXP New trial (rep=0, index=1): <Trial 1 (1 in rep 0) data={'direction': 'right', 'position': [0.25, 0], 'thisN': 1, 'thisTrialN': 1, 'thisRepN': 0}>
2.7436 EXP trial_target: autoDraw = False
2.7436 EXP text: autoDraw = False
2.7436 EXP ISI_dot: autoDraw = True
2.7460 DATA Keypress: left
```

Feedback

Instructions,
[random ISI, either stimulus “left” or “right” on either left or right,
and a keypress, followed by feedback “correct” “incorrect”] <-repeat,
end

feedback_text Properties

Basic Layout Appearance Formatting Data Testing

Name

Start \$ time (s) 0.0
Expected start (s)

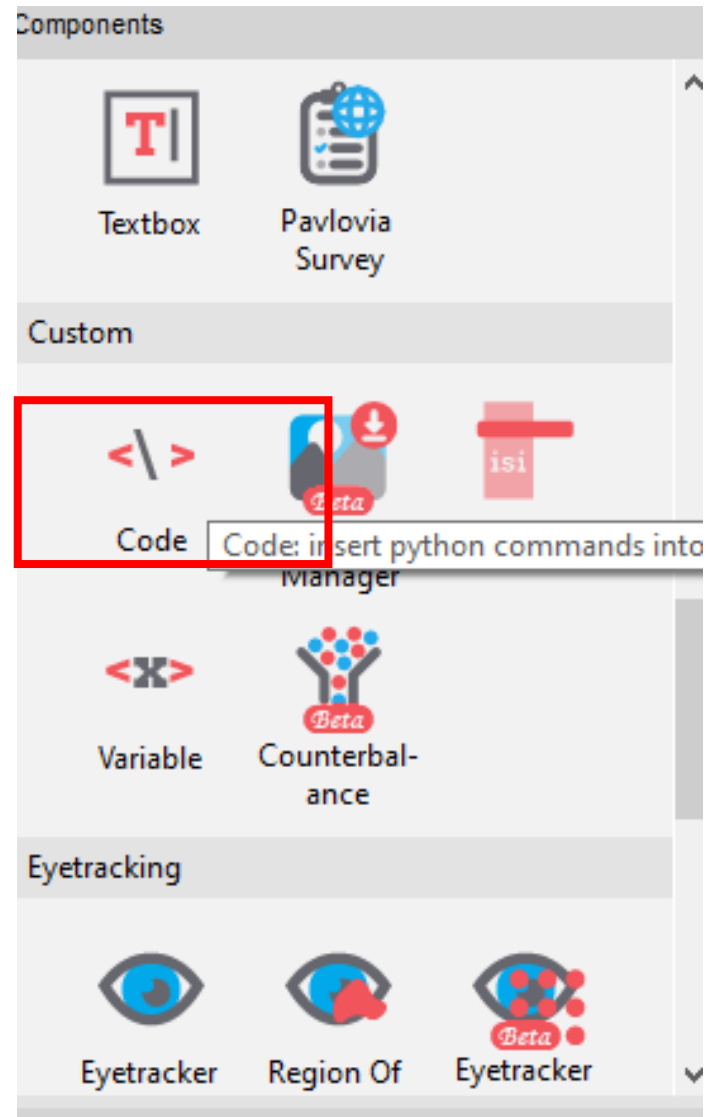
Stop \$ duration (s) 1.0
Expected duration (s)

Disable Component ☐

Text

set every repeat

Help OK Cancel



Name Code type ☐ Disabled☐ Before Exp. ☐ Begin Exp. ☐ Begin Routine ☐ Each Frame ☒ End Routine * ☐ End Exp.

```
1 # Check if a response was actually made
2 if trial_key_resp.keys:
3     # Compare the response to your Excel column named '
4     if trial_key_resp.keys == corrAns:
5         thisExp.addData('accuracy', 1)
6         feedback_message = "Correct!"
7     else:
8         thisExp.addData('accuracy', 0)
9         feedback_message = "Incorrect"
10 else:
11     # If they took too long and the trial timed out
12     thisExp.addData('accuracy', None)
13     feedback_message = "Too Slow!"
```

```
1 if (trial_key_resp.keys) {
2     if ((trial_key_resp.keys === corrAns)) {
3         psychoJS.experiment.addData("accuracy", 1);
4         feedback_message = "Correct!";
5     } else {
6         psychoJS.experiment.addData("accuracy", 0);
7         feedback_message = "Incorrect";
8     }
9 } else {
10     psychoJS.experiment.addData("accuracy", null);
11     feedback_message = "Too Slow!";
12 }
13
```

AE	AF	AG	AH	AI	AJ	AK	AL	AM	
xt.starte	trial.stopp	trials.trial	trials.trial	trials.trial	accuracy	feedback.	feedback_	feedback.	p
.495084	3.096963	left	0.574395	None	1	3.105098	3.128961	4.09765	
.164426	5.800557	right	0.634646	None	1	5.800856	5.832383	6.833309	
.201788	8.769666	left	0.553492	None	0	8.775068	8.801587	9.770292	
1.00432	11.33854	right	0.303166	None	0	11.33879	11.37057	12.37214	
3.53979	14.3752	right	0.82772	None	1	14.37986	14.40766	15.40904	

Challenge #3

- Use AI to change the code block so that:
 - “Correct!” is green
 - “Incorrect!” is red
- Use AI to add code block so that:
 - A smiley face is presented as correct feedback
 - A frowny face is presented as incorrect feedback

Practice

Extracted from the workshop “Creating an Experiment in PsychoPy”
(<https://workshops.psychopy.org/>)

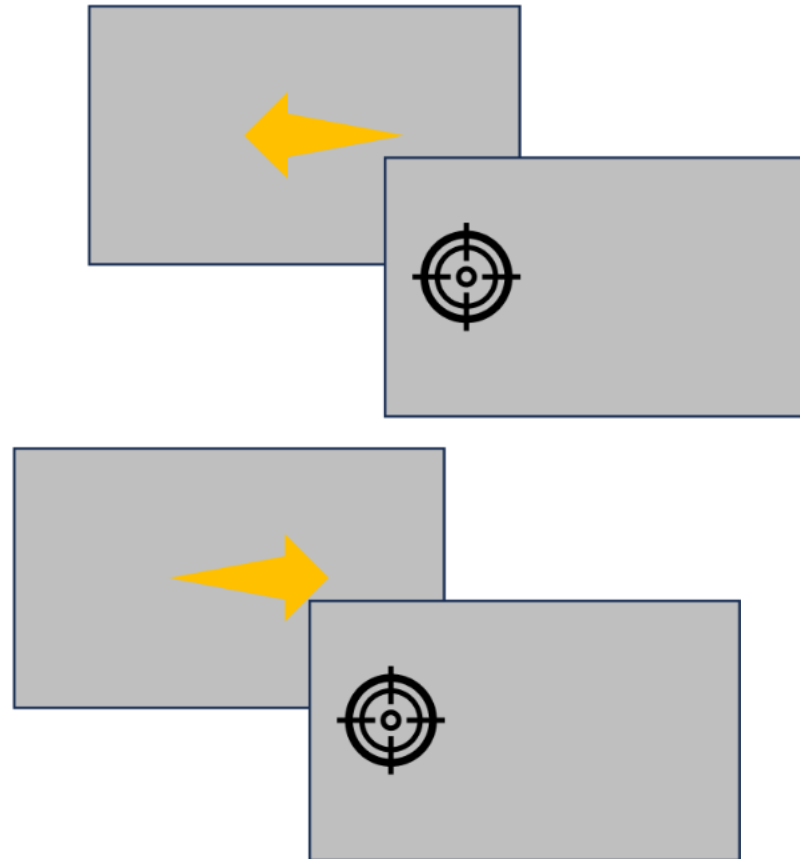
Setting up your trial

Example: A [Posner cueing task](#). An arrow cues the location of a target. The participant must respond using the left and right arrow keys.

What component make up this routine?

- An arrow/cue
- A target
- A keyboard response (left/right keypress - respond to the location of the target)

Example trials valid (top) Invalid trial (bottom)

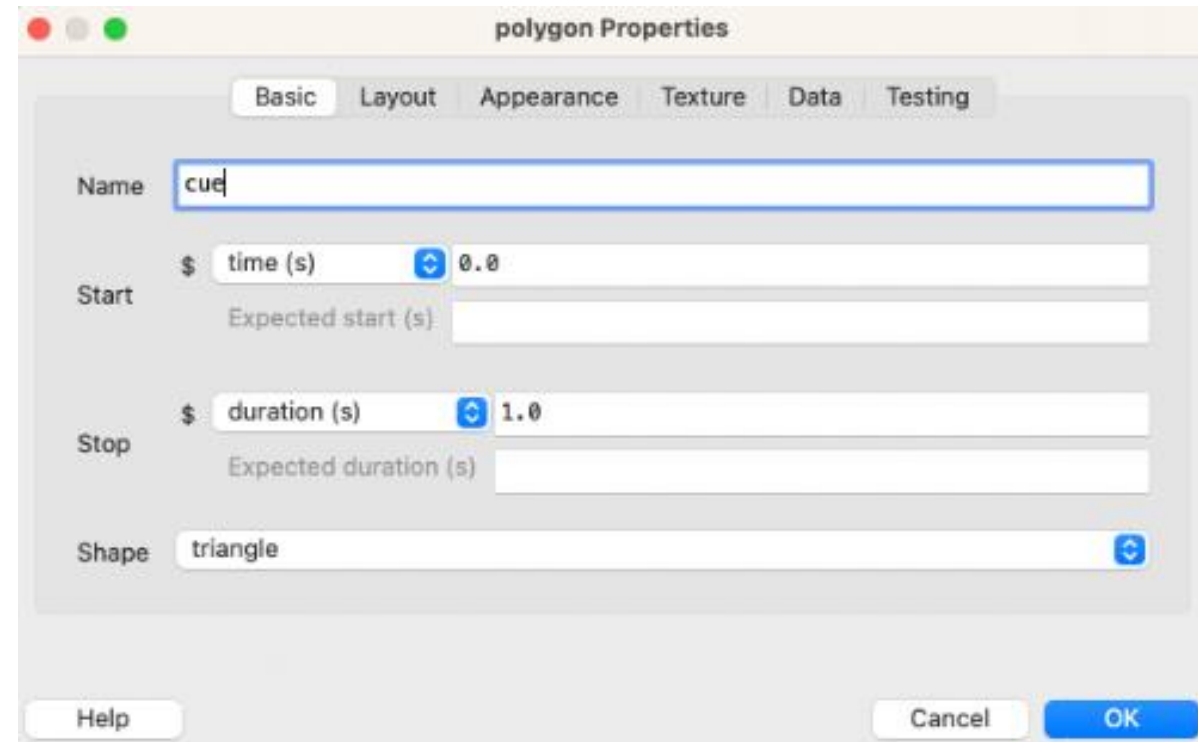


Presenting Shapes: Add a “Cue”

Let's add our cue to the "trial" routine.

Cue stimulus: Add a **polygon** and leave the shape as "triangle". The cue starts at 0 seconds and lasts 1 second. in the **layout** tab set **orientation** to 90.

In the layout tab make the cue slightly smaller by setting size to (0.1, 0.1)



The screenshot shows the 'polygon Properties' dialog box with the 'Basic' tab selected. The 'Name' field is set to 'cue'. The 'Start' section shows '\$ time (s)' set to 0.0. The 'Stop' section shows '\$ duration (s)' set to 1.0. The 'Shape' dropdown is set to 'triangle'. The 'Layout' tab is visible but not selected. The 'Expected start (s)' and 'Expected duration (s)' fields are empty. The 'Help', 'Cancel', and 'OK' buttons are at the bottom.

Property	Value
Name	cue
Start (\$ time (s))	0.0
Expected start (s)	
Stop (\$ duration (s))	1.0
Expected duration (s)	
Shape	triangle

Presenting Shapes: Add a “Target”

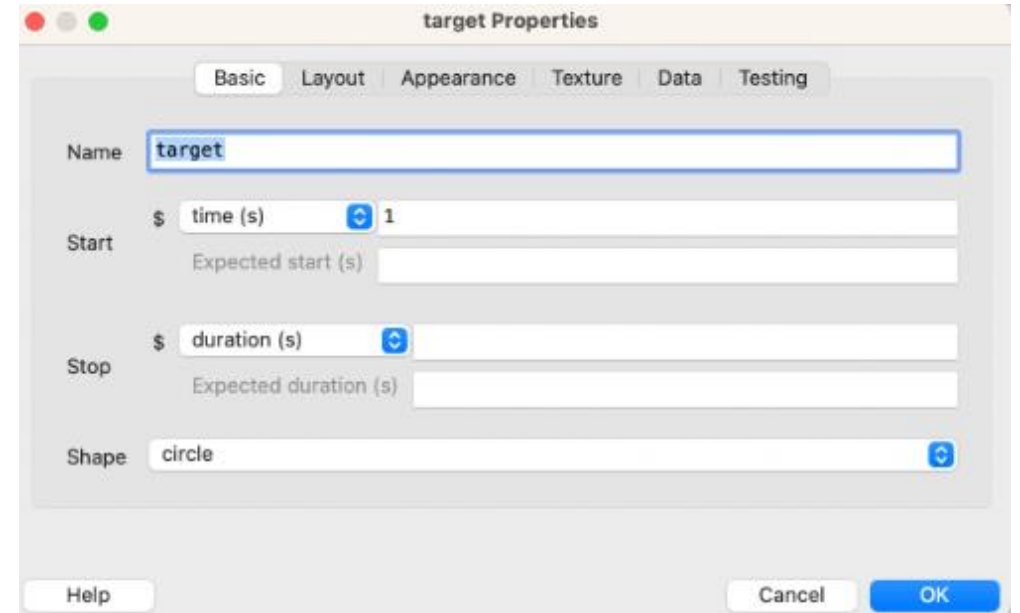
Let's add our target to the "trial" routine.

Target stimulus: Add another **polygon**, set shape to be "circle". The target starts at 1 second and stays onscreen until a response is made.

To make a stimulus stay on screen until a response is made, leave the duration as blank. In the **layout** tab set the **position** to (0.5, 0).

Make the target slightly smaller by setting size to (0.1, 0.1)

 The x, y coordinate for the centre of your screen is 0, 0. Positive x values are right and negative left. Positive y values are up and negative down. Therefore a position of (0.5, 0) will position your target on the right of the screen.



target Properties

Basic Layout Appearance Texture Data Testing

Name: target

Start: \$ time (s) 1
Expected start (s)

Stop: \$ duration (s)
Expected duration (s)

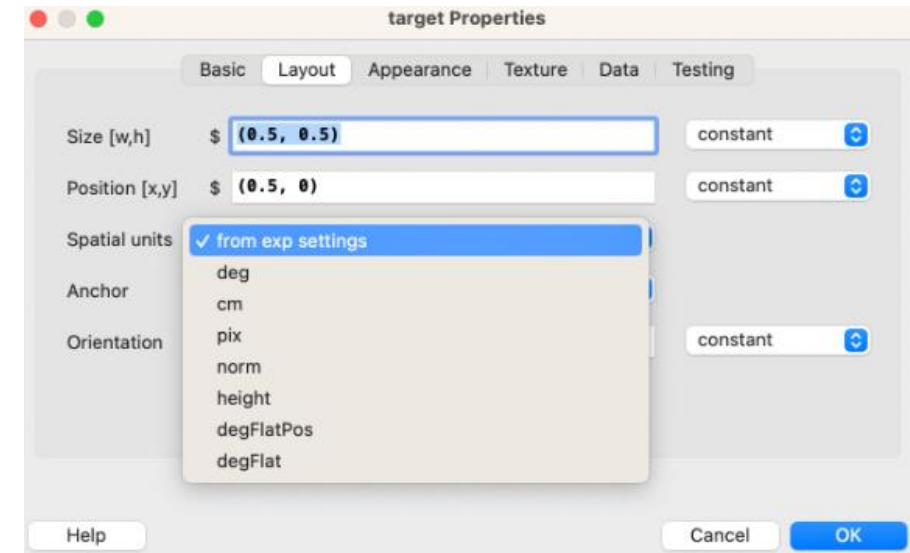
Shape: circle

Help Cancel OK

Understanding PsychoPy units

When controlling the size and position of your stimuli, you want to know what unit stimuli are presented in. You can see what unit your stimuli are using in the **layout** tab. You can see more about PsychoPy units [here](#).

- **from exp settings:** unit inherited from global experiment settings (see next slide). By default we use "height". This means that everything will be scaled proportionally to the window size. a size of 0.5 corresponds to 50% of the window height.



If you are using any unit other than "height" or "norm" you must configure your monitor settings in the Monitor Centre. For online experiments, we recommend "height" units because this means that stimuli will scale proportionally to the participants screen.



Your Experiment Settings

By selecting the cog icon at the top of the Builder app, you can access the global settings of your experiment.

In the "Basic" tab you can configure your experiment name, what version of PsychoPy you are using and add/remove fields in the "Experiment Info".

In the "Screen" tab you can set up the global units used in your experiment, if you want to use full screen, and which monitor from your [Monitor Centre](#) you want to use.



Everything in the "Experiment Info" will be presented as options in a dialogue box at the start of your experiment. So you can provide info such as participant ID.

Field	Default		
participant	f"{randint(0, 999999):06.0f}"	+	-
session	001	+	-

Buttons at the bottom: Help, Cancel, OK

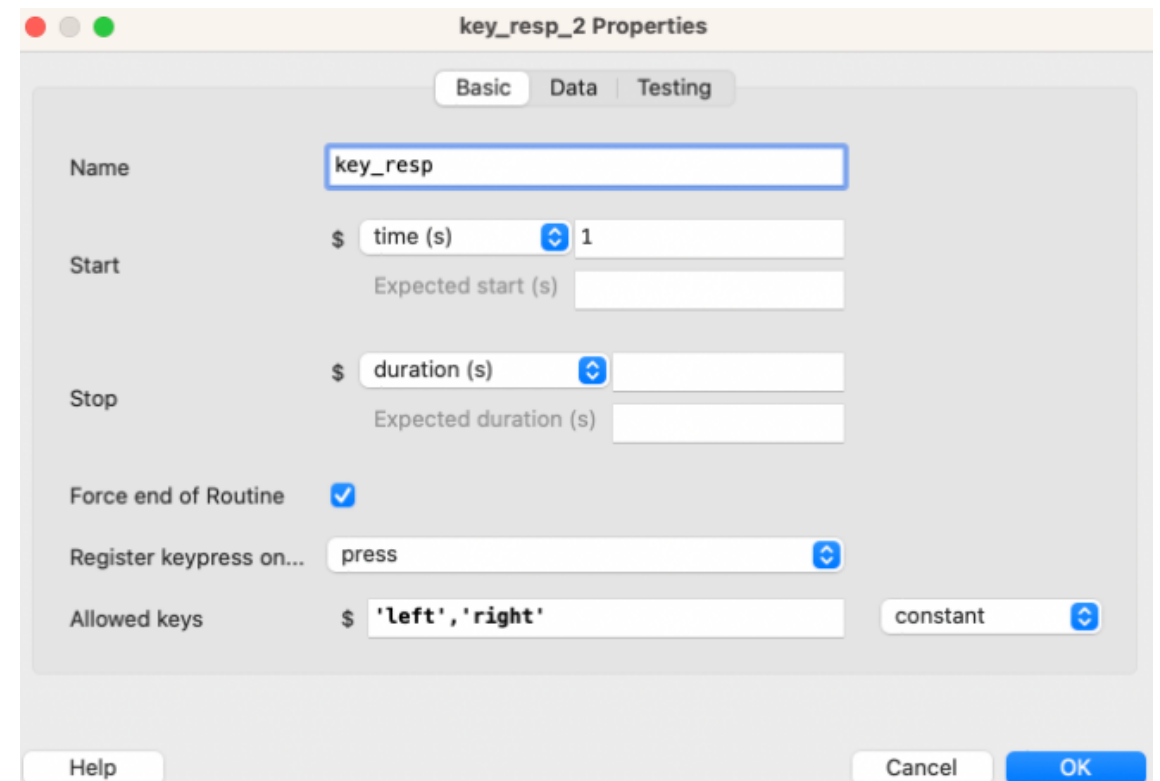
Gather a Keyboard Response

Add a keyboard component to the trial routine, set the start time to 1 (start of the target) and allow “left” and “right” keys.

Keyboard responses also have a start time and duration, by default, the duration field is blank (meaning it will last indefinitely).

Force end of Routine: If checked, a keyboard response will end this routine, and the experiment will move on.

Allowed keys: A list of key names to watch (each name must be in quotations and separated by commas). if blank, all keys will be watched.



The screenshot shows a dialog box titled "key_resp_2 Properties" with three tabs: "Basic", "Data", and "Testing". The "Basic" tab is selected. The dialog contains the following fields and controls:

- Name:** A text field containing "key_resp".
- Start:** A section with a "\$" icon, a dropdown menu set to "time (s)", a numeric input field containing "1", and a sub-field labeled "Expected start (s)" which is empty.
- Stop:** A section with a "\$" icon, a dropdown menu set to "duration (s)", an empty numeric input field, and a sub-field labeled "Expected duration (s)" which is empty.
- Force end of Routine:** A checkbox that is checked.
- Register keypress on...:** A dropdown menu set to "press".
- Allowed keys:** A section with a "\$" icon, a text field containing "'left', 'right'", and a dropdown menu set to "constant".

At the bottom of the dialog are three buttons: "Help", "Cancel", and "OK".

Check it works!

You can run your experiment by pressing the green "run" icon. You should see your cue appear for 1 second, then your target, which should remain onscreen until a response is made.

Setting up your trials spreadsheet

Most experiments consist of a **series of trials**. Before making an experiment think about what changes trial by trial and make a spreadsheet.

What changes trial-by-trial?

- Arrow orientation (90 or 270 degrees)
- Target location (x coordinate)
- Correct answer (left or right)

	A	B	C	D
1	arrow_orientation	target_x_location	correct_keypress	condition
2	90	0.5	right	valid
3	270	-0.5	left	valid
4	90	-0.5	left	invalid
5	270	0.5	right	invalid
6				

Column headers: Parameters that change trial-by-trial.

Row values: The value of each parameter on each trial.

Repeating a routine e.g. a series of trials

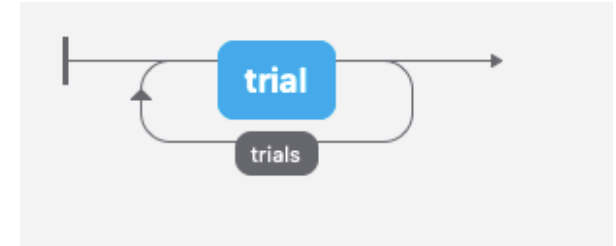
Select "Insert Loop" and select either side of the routine(s) you want to repeat (for us this is the trial routine)

In the conditions field select the spreadsheet we created.

Conditions: A spreadsheet detailing what changes trial-by-trial (see previous slide)

nReps: the number of times the spreadsheet rows are repeated (if no spreadsheet this is just the number of times the loop repeats)

Selected Rows: Row indices from the spreadsheet to present



The 'Loop Properties' dialog box is shown with the following fields and values:

- Name: trials
- loopType: random
- Is trials: ☒
- nReps: \$ 5
- Selected rows: (empty field)
- random seed: \$ (empty field)
- Conditions: myspreadsheet.csv

At the bottom right, there is a green icon and the text 'No parameters set'. At the bottom, there are three buttons: 'Help', 'Cancel', and 'OK'.

Repeating a routine e.g. a series of trials

Loop Properties

Name: trials

loopType: random

Is trials: ☒

nReps: \$ 5

Selected rows:

random seed: \$

Conditions: myspreadsheet.csv

No parameters set

Help Cancel OK

loopType:

- **random** - randomise rows of spreadsheet - within repetition.
- **full random** - randomise across repetitions of the spreadsheet.
- **staircase/interleaved staircase** - specialised adaptive procedures used typically in psychophysics.

Is Trials: if yes, add new row to data file for every repeat of this loop.

i If you set **nReps to 0** then everything inside that loop will be skipped. This is useful for debugging and branched designs (e.g. if you only want to show part of your experiment)

Updating stimuli trial-by-trial

We can now update our stimuli using the variables in our spreadsheet.

In the **target** stimulus, we can update the x coordinate using **target_x_location**.

	A	B	C	D
1	arrow_orientation	target_x_location	correct_keypress	condi
2	90		0.5 right	valid
3	270		-0.5 left	valid
4	90		-0.5 left	invalid
5	270		0.5 right	invalid

target Properties

Basic Layout Appearance Texture Data Testing

Size [w,h] \$ (0.5, 0.5) constant

Position [x,y] \$ (target_x_location, 0) set every repeat

Spatial units from exp settings

Anchor center

Orientation \$ 0 constant

Draggable? ☐

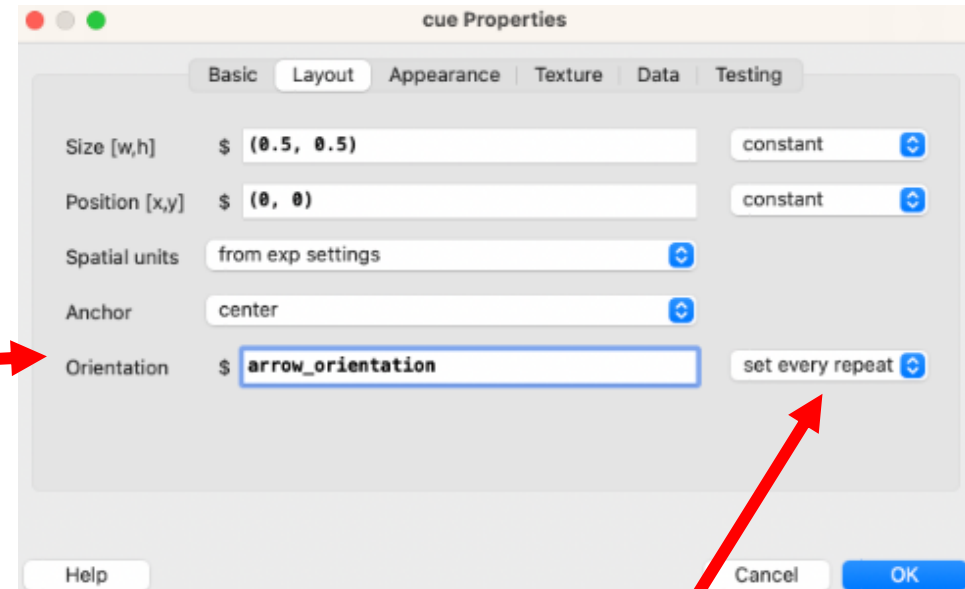
Set the field to "set every repeat"

Updating stimuli trial-by-trial

We can now update our stimuli using the variables in our spreadsheet.

In the **cue** stimulus we update the orientation using **arrow_orientation**.

	A	B	C	D
1	arrow_orientation	target_location	correct_keypress	condition
2	90		0.5 right	valid
3	270		-0.5 left	valid
4	90		-0.5 left	invalid
5	270		0.5 right	invalid



i If you are using a variable to update a component property start the field with a '\$' sign only if that field does not already have a '\$' in it.s field.

Set the field to "set every repeat"

Check it works!

You can run your experiment by pressing the green "run" icon. You should now see the location of your target and the orientation of your cue change trial-by-trial.

Exercise

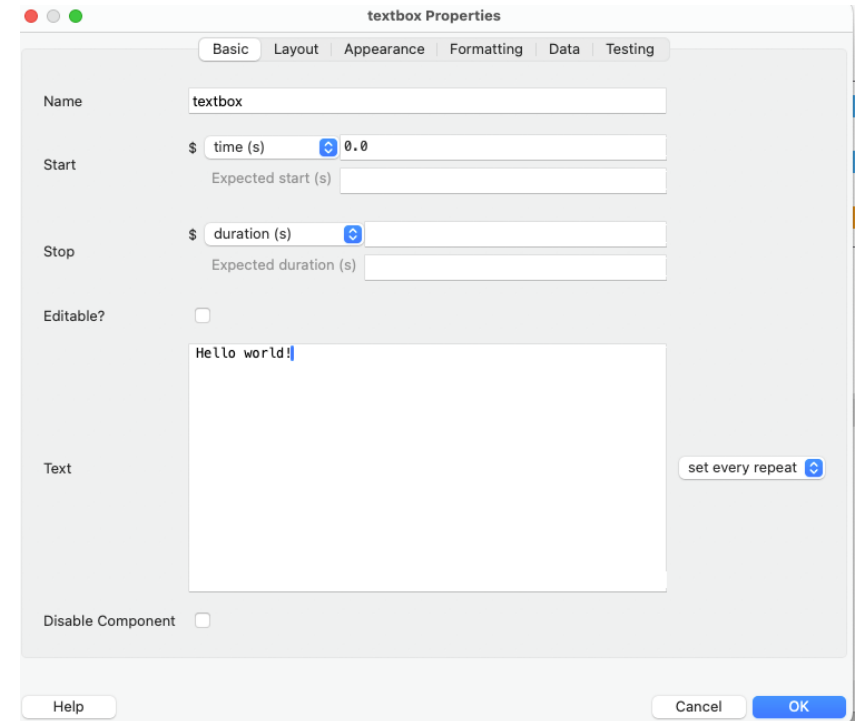
- Add a new routine for your instructions
- Present some text telling the participant what to do, allow them to press space to start the experiment.
- Add a routine to say "Thanks" at the end of the experiment.
- Add another field to your experiment settings to gather more information (e.g. handedness, age)

Presenting Text

Two components can be used for adding text. **Text** and **TextBox**. TextBox has the added benefit that you can use similar formatting to what you would with a text box e.g. anchor alignment.



You can also use the TextBox component to gather typed responses

A screenshot of a software interface titled 'textbox Properties'. The dialog has several tabs: 'Basic', 'Layout', 'Appearance', 'Formatting', 'Data', and 'Testing'. The 'Basic' tab is selected. It contains the following fields and controls:

- Name:** A text field containing 'textbox'.
- Start:** A section with a '\$' icon, a 'time (s)' label, a spinner set to '0.0', and an 'Expected start (s)' text field.
- Stop:** A section with a '\$' icon, a 'duration (s)' label, a spinner, and an 'Expected duration (s)' text field.
- Editable?:** A checkbox that is currently unchecked.
- Text:** A large text area containing the text 'Hello world!'. To the right of this area is a button labeled 'set every repeat' with a dropdown arrow.
- Disable Component:** A checkbox at the bottom left that is unchecked.

At the bottom of the dialog are three buttons: 'Help', 'Cancel', and 'OK'.

Storing accuracy from keypress responses

To store whether a participant was correct or incorrect, you can use the column from your trials spreadsheet.

1. In your keyboard component select the "Data" tab.
2. Check "store correct"
3. in the correct answer field feed in the column from your spreadsheet.

	A	B	C	D
1	arrow_orientation	target_x_location	correct_keypress	condition
2	90		0.5 right	valid
3	270		-0.5 left	valid
4	90		-0.5 left	invalid
5	270		0.5 right	invalid

key_resp Properties

Basic Data Testing

Store last key

Store correct ☒

Correct answer \$correct_keypress

Save onset/offset times ☒

Sync timing with screen ☒

Discard previous ☒

Help Cancel OK

Extending Builder with code

Sometimes you might want to add flexibility to your experiment in ways that might not be accommodated through the GUI interface. This is one of the reasons people may prefer to use code.

In PsychoPy, you can use code components to add flexibility where code is needed.

Example: I want to give my participant trial-by-trial feedback (e.g. "**Correct!**" if response was correct and "**Incorrect**" if response was incorrect). In this example we do not know what the text we present will be until a response is made, so we need some conditional IF/ELSE statement to set up our text.

Adding trial-by-trial feedback

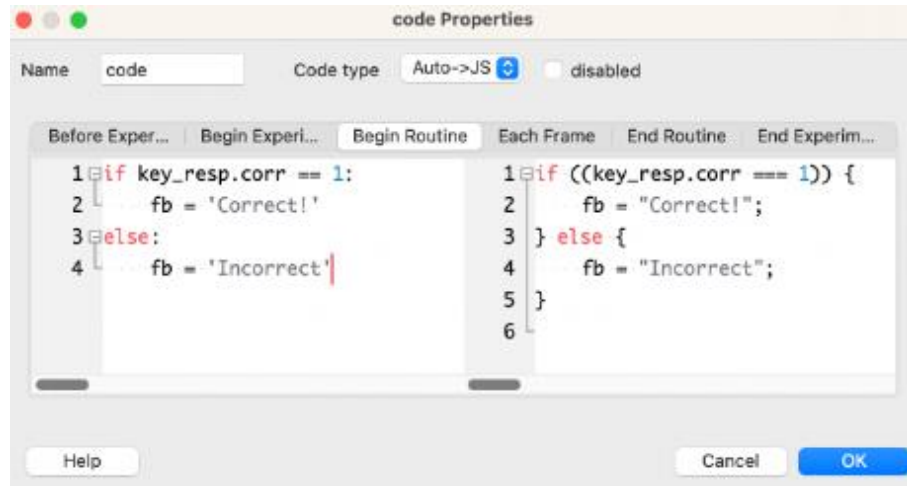
Start by adding a new routine to your experiment called "feedback". It should appear after the "trial" routine, but it should occur on every trial, so it therefore needs to be within the loop.



Adding trial-by-trial feedback

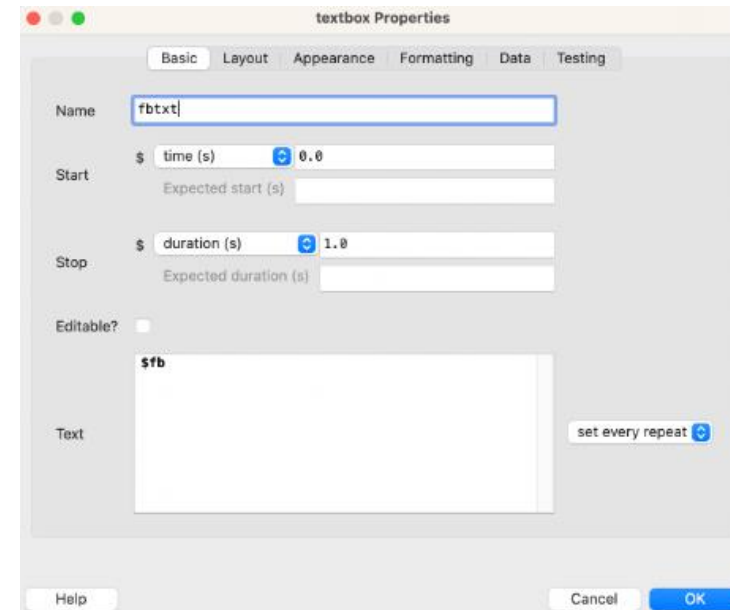
We then add two components to our "feedback" routine. A code component (in the "custom" section of Components) and a TextBox.

⚠ The tab you add your code to is important. Here we don't know if you are correct until the beginning of your feedback routine. So we add the code there.



Code component

The code component checks if the answer was correct by inspecting your `key_resp` component, which now has the property `corr`. This will be 1 if correct and 0 if incorrect. So we use this to make a variable `fb`.



Text box component

We use the variable `fb` as the text in our TextBox component, make sure to **set every repeat**.

Go/no-go task

Mouse click

Trial feedback

Block feedback

Skipping routine

Questionnaire

Challenge #4

- Create your own task!
- Come up with a proper research question / hypothesis that you can answer using a task, e.g.,
 - RT for stimulus A > stimulus B > Stimulus C, etc.
 - Accuracy in condition X < condition Y, etc.
 - Variable V is related to variable W, etc.

Resources

File

Unpack Demos...

Browse on Pavlovia

Open demos folder

Pilot

Run

Experiment

Desktop

Browser

No user

No project

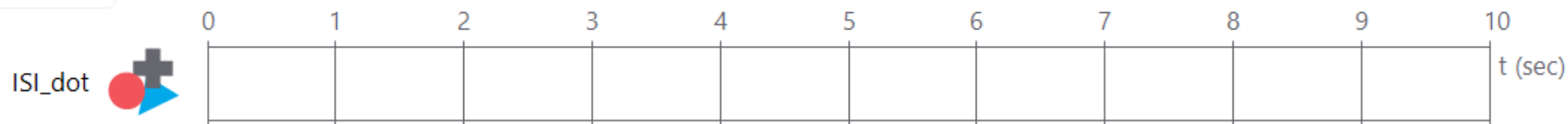
Pavlovia

Views

Routines

intro trial end ISI x

⚙ Routine settings



Components

+ Get more...

Favorites



Image



Keyboard



Mouse



Slider



Sound



Text

Stimuli



Aperture



Polygon



Dots



Grating



Image

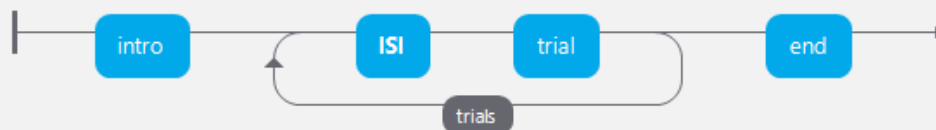


Movie

Flow

Insert Routine

Insert Loop



<https://psychopy.org/index.html>

<https://psychopy.org/PsychoPyManual.pdf>



**JONATHAN PEIRCE, REBECCA HIRST
& MICHAEL MACASKILL**

BUILDING EXPERIMENTS IN

Psychopy

2ND EDITION

